CEN454/Intelligent systems
Sep 2024

Project:
UCMerced Land use Classification

Talal Aloushan
441102306

Introduction:
In this project im going to train a CNN model(both Pytorch&Tensorflow frameworks) on the UCMerced dataset which consists of 21 class each with 100 images with a total of 2100 , to get hands-on experience of Deep learning and computer vision, first of all i have splitted the data into training set(60%) and validation set(40%), then i applied data augmentation on the training set,i choose the EffecientnetB0 (imagenet) model which has 4 Million parameters to train it on my data, at first i froze the whole model weights and added an output dense layer with 21 neurons, so im training the last layers weights only, i got 96% accuracy, then i unfroze some of the deep layers, 20% of the model only, and started training, i got 97%, then i switched to the Yolov11n-cls model which has 1.5 Million parameters and got 98% accuracy.

Dataset explore:

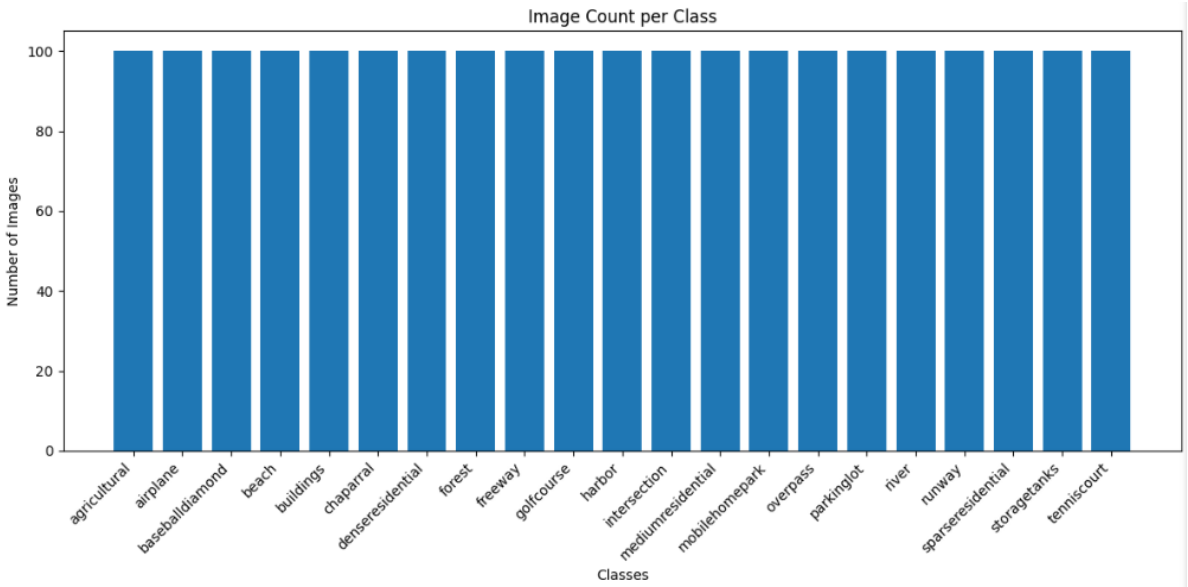
This is a 21 class land use image dataset meant for research purposes.

There are 100 images for each of the following classes:

- agricultural
- airplane
- baseballdiamond
- beach
- buildings
- chaparral
- denseresidential
- forest
- freeway
- golfcourse
- harbor
- intersection
- mediumresidential
- mobilehomepark
- overpass
- parkinglot
- river
- runway
- sparseresidential
- storagetanks
- tenniscourt

Each image measures 256x256 pixels.

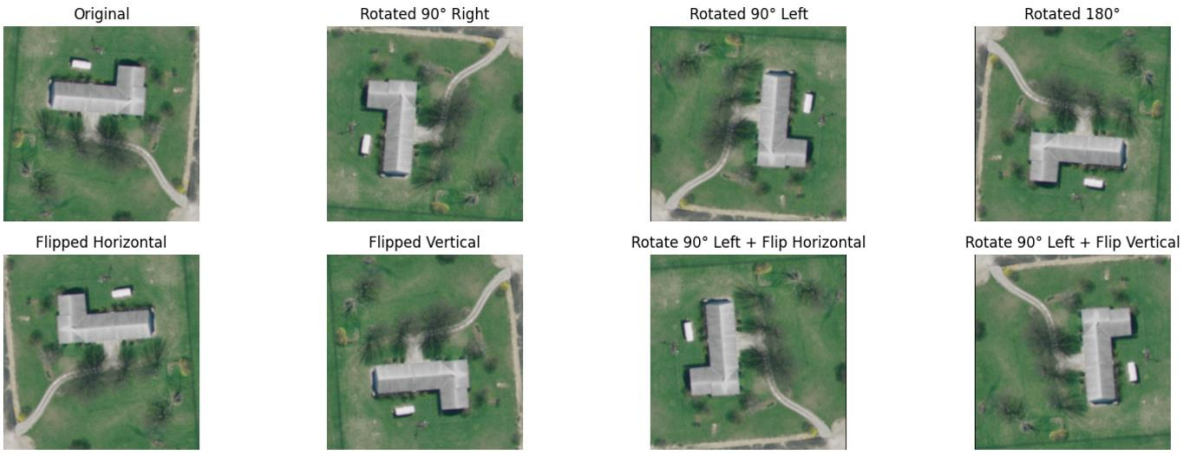
The images were manually extracted from large images from the USGS National Map Urban Area Imagery collection for various urban areas around the country. The pixel resolution of this public domain imagery is 1 foot.


Image Count per Class
Number of Images
0
20
40
60
80
100
agricultural
airplane
baseballdiamond
beach
buildings
chaparral
denseresidential
forest
freeway
golfcourse
harbor
intersection
mediumresidential
mobilehomepark
overpass
parkinglot
river
runway
sparseresidential
storagetanks
tenniscourt
Classes

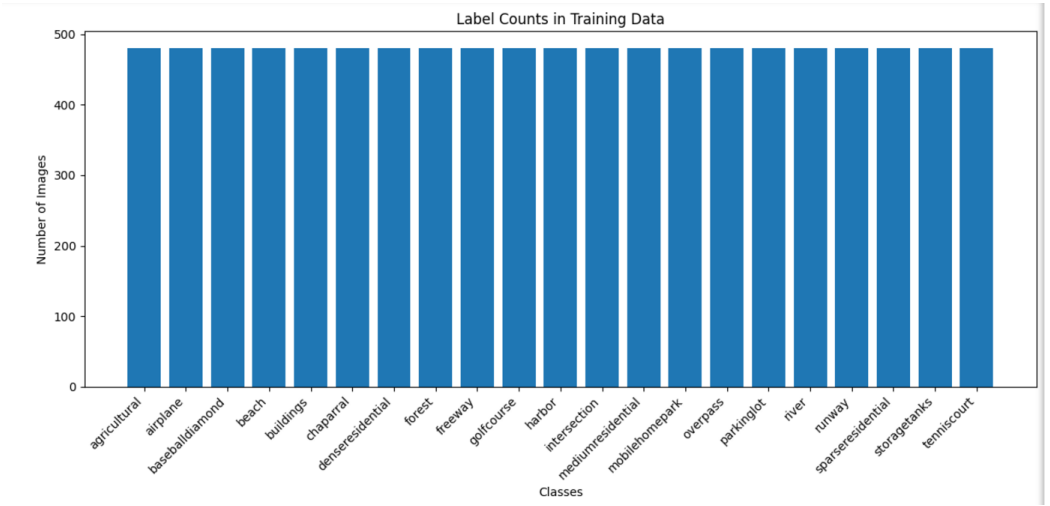

agricultural
airplane
baseballdiamond
beach
buildings
chaparral
denseresidential
forest
freeway
golfcourse
harbor
intersection
mediumresidential
mobilehomepark
overpass
parkinglot
river
runway
sparseresidential
storagetanks
tenniscourt

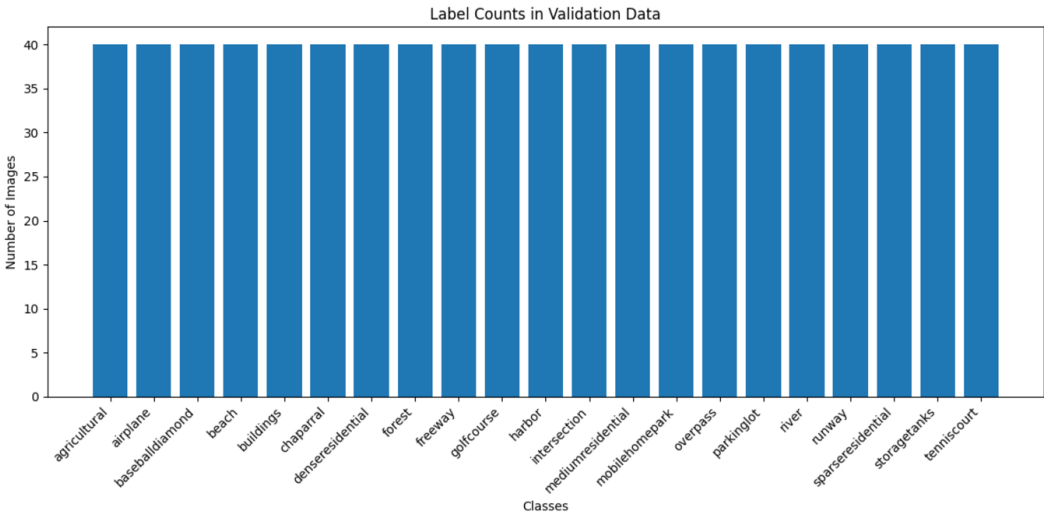
Data split and augmentation:

I splitted the data into 60% 40%, did 7 augmentation which is :



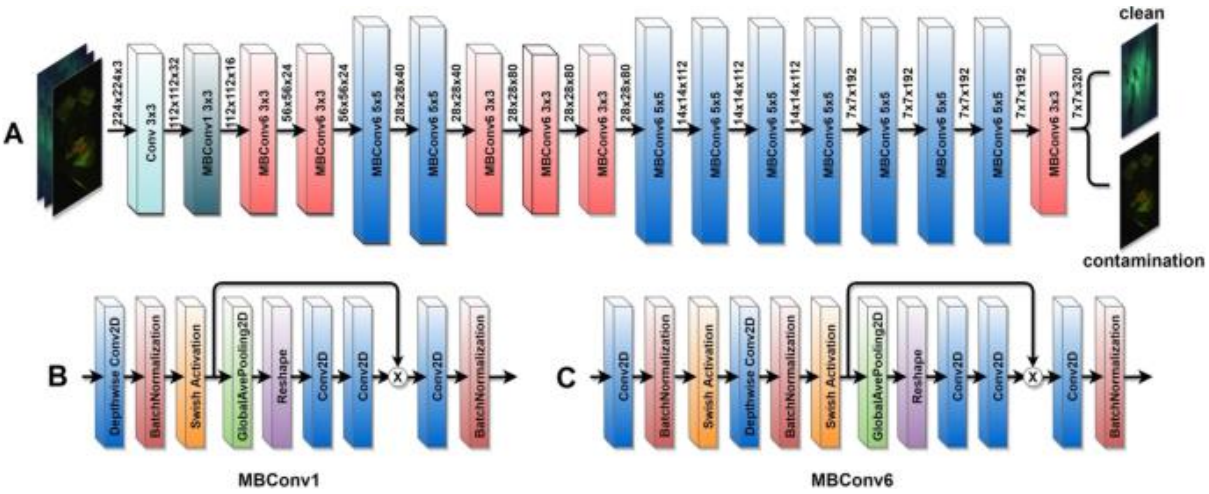
New label count chart:





480 x 21 = 10080 training images.
40 x 21 = 840 validation images.

CNN Model (EfficentnetB0)imagenet based:



Approach#1(Tensorflow) (frozen model)(96%acc):

Data preprocessing:
Batched data into 16 batches
Scaled pixel values in the range (0,1)
Resized to 224x224

So i froze the entire model and added a dropout layer to prevent overfitting and a dense output layer with 21 nodes:

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 8, 8, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 21)	26,901

Total params: 4,076,472 (15.55 MB)

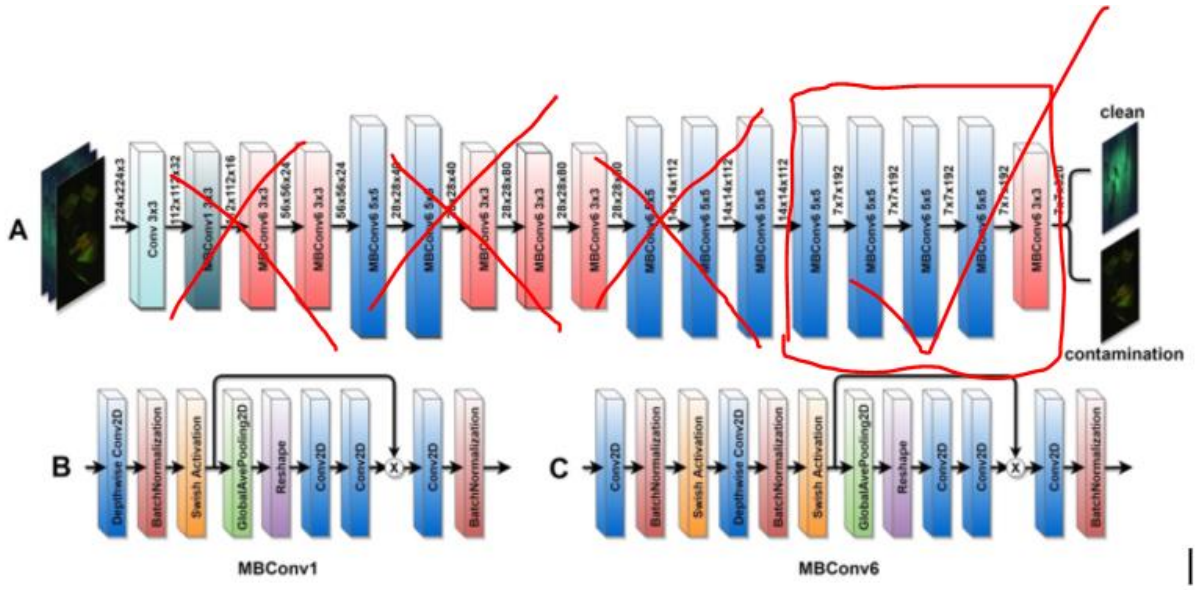
Trainable params: 26,901 (105.08 KB)

Hyperparameters:

```
model.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(learning_rate=0.001),
    metrics=['accuracy']
)
```

```
630/630 ━━━━━━━━━━━━━━━━━━━━ 296s 455ms/step - accuracy: 0.7653 - loss: 0.9875 - val_accuracy: 0.9333 - val_loss: 0.2491
Epoch 2/10
630/630 ━━━━━━━━━━━━━━━━━━━━ 292s 463ms/step - accuracy: 0.9773 - loss: 0.1291 - val_accuracy: 0.9488 - val_loss: 0.1731
Epoch 3/10
630/630 ━━━━━━━━━━━━━━━━━━━━ 287s 455ms/step - accuracy: 0.9884 - loss: 0.0729 - val_accuracy: 0.9607 - val_loss: 0.1511
Epoch 4/10
630/630 ━━━━━━━━━━━━━━━━━━━━ 279s 442ms/step - accuracy: 0.9944 - loss: 0.0494 - val_accuracy: 0.9607 - val_loss: 0.1431
Epoch 5/10
630/630 ━━━━━━━━━━━━━━━━━━━━ 280s 444ms/step - accuracy: 0.9953 - loss: 0.0361 - val_accuracy: 0.9607 - val_loss: 0.1349
```

Approach#2(Tensorflow) (unfroze block6 and upper layers)(97%acc):



Actually not all the block6, i unfroze from block6d_se_excite layers exactly, which is 20% of block6 .

Layer 217: block6d_se_excite

Trainable params: 1,369,669 (5.22 MB)

Changed learning rate from 0.001 to 0.0001

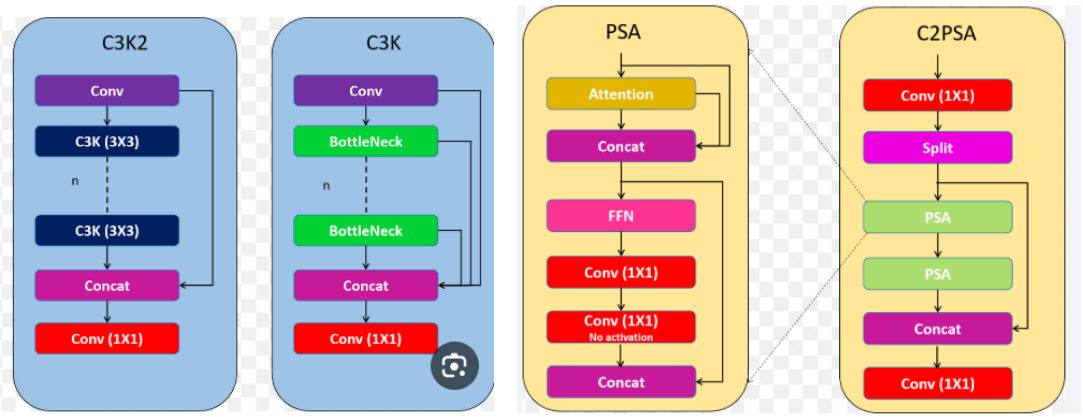
```
Epoch 1/20
630/630 ━━━━━━━━━━━━━━━━━━━━ 314s 483ms/step - accuracy: 0.9930 - loss: 0.0262 - val_accuracy: 0.9667 - val_loss: 0.1235
Epoch 2/20
630/630 ━━━━━━━━━━━━━━━━━━━━ 305s 485ms/step - accuracy: 0.9967 - loss: 0.0116 - val_accuracy: 0.9726 - val_loss: 0.1271
Epoch 3/20
630/630 ━━━━━━━━━━━━━━━━━━━━ 308s 489ms/step - accuracy: 0.9993 - loss: 0.0039 - val_accuracy: 0.9631 - val_loss: 0.1891
Epoch 4/20
630/630 ━━━━━━━━━━━━━━━━━━━━ 315s 499ms/step - accuracy: 0.9968 - loss: 0.0105 - val_accuracy: 0.9714 - val_loss: 0.1393
Epoch 5/20
630/630 ━━━━━━━━━━━━━━━━━━━━ 306s 485ms/step - accuracy: 0.9992 - loss: 0.0032 - val_accuracy: 0.9714 - val_loss: 0.1565
```

Next I tried to unfreeze until block 4 but my cpu and memory can't handle it. Anyway i think freezing the model with approach#1 was the best as it has only 26k trainable parameters and it performs nearly as the semi-unfrozen model that has 1.3Million trainable parameters. I will need to increase the epochs to get higher accuracy.

Approach#3(Pytorch)(Yolo11n-cls)imagenetbased(98%acc):
In this approach im fine-tuning all the weights of the model

	from	n	params	module	arguments	0
-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]	1	
-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]	2	
-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]	3	
-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]	4	
-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]	5	
-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]	6	
-1	1	87040	ultralytics.nn.modules.block.C3k2	[128, 128, 1, True]	7	
-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]	8	
-1	1	346112	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True]	9	
-1	1	249728	ultralytics.nn.modules.block.C2PSA	[256, 256, 1]	10	
-1	1	357141	ultralytics.nn.modules.head.Classify	[256, 21]		

YOLO11n-cls summary: 151 layers, 1,558,005 parameters, 1,558,005 gradients, 3.3 GFLOPs



Data preprocessing:
Batched data into 16 batches
Scaled pixel values in the range (0,1)
Resized to 224x224

optimizer: AdamW(lr=0.000714, momentum=0.9) with parameter groups 39 weight(decay=0.0), 40 weight(decay=0.0005), 40 bias(decay=0.0)

```
Epoch    GPU_mem       loss  Instances       Size
11/20         0G    0.04048         16        224: 100%|██████████| 630/630 [03:03<00:00,  3.43it/s]
          classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:05<00:00,  5.22it/s]
              all      0.971          1

Epoch    GPU_mem       loss  Instances       Size
12/20         0G    0.03342         16        224: 100%|██████████| 630/630 [03:03<00:00,  3.44it/s]
          classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:05<00:00,  5.24it/s]
              all      0.975          1

Epoch    GPU_mem       loss  Instances       Size
13/20         0G    0.03429         16        224: 100%|██████████| 630/630 [03:02<00:00,  3.44it/s]
          classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:05<00:00,  5.23it/s]
              all      0.975          1

Epoch    GPU_mem       loss  Instances       Size
14/20         0G    0.03263         16        224: 100%|██████████| 630/630 [05:17<00:00,  1.99it/s]
          classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:05<00:00,  5.16it/s]
              all      0.982          1

Epoch    GPU_mem       loss  Instances       Size
15/20         0G    0.02439         16        224: 100%|██████████| 630/630 [03:03<00:00,  3.44it/s]
          classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:05<00:00,  5.27it/s]
              all      0.982          1

Epoch    GPU_mem       loss  Instances       Size
16/20         0G    0.02283         16        224: 100%|██████████| 630/630 [03:02<00:00,  3.45it/s]
          classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:05<00:00,  5.26it/s]
              all      0.973          1

Epoch    GPU_mem       loss  Instances       Size
17/20         0G    0.01789         16        224: 100%|██████████| 630/630 [03:02<00:00,  3.45it/s]
          classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:05<00:00,  5.24it/s]
              all      0.981      0.999

Epoch    GPU_mem       loss  Instances       Size
18/20         0G    0.01591         16        224: 100%|██████████| 630/630 [05:14<00:00,  2.00it/s]
          classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:11<00:00,  2.39it/s]
              all      0.985      0.999
```

```
test: C:\Users\HP\yolo11-cls\UCMerced_augmented\test... found 840 images in 21 classes
               classes   top1_acc   top5_acc: 100%|██████████| 27/27 [00:10<00:00,  2.58it/s]
                   all      0.985      0.999
```

yolo11n-cls default random augmentations during training:

